



西北工业大学  
NORTHWESTERN POLYTECHNICAL UNIVERSITY

# 数字电子技术基础

## 实验报告

### 实验四：基于硬件描述语言电路设计

---

课程： 数字电子技术实验

组员 1: 郭浩然 2024303980

组员 2: 庞铭洋 2024302539

组号: 15

时间: 2026 年 5 月 25 日

---

## 1 实验目的

1. 掌握 VHDL 硬件描述语言的基本语法，理解实体 (Entity)、结构体 (Architecture)、信号 (Signal) 和进程 (Process) 等核心概念。
2. 学习使用 VHDL 分别设计组合逻辑电路 (异或门、七段码译码器) 和时序逻辑电路 (计数器、分频器)，并在 Quartus II 中进行编译和波形仿真验证。
3. 掌握 Quartus II 中 VHDL 文件的编译、生成原理图模块符号 (Create Symbol Files) 以及在顶层原理图中调用 VHDL 模块的基本流程。
4. 能够将多个独立设计的 VHDL 模块在顶层原理图中组合连接，实现较复杂的数字系统设计，并下载到 DE0 开发板进行硬件验证。

## 2 实验要求

1. **要求 1:** 学习并掌握硬件描述语言 VHDL，熟悉门电路的逻辑功能，并用硬件描述语言实现门电路的设计。参考“参考内容 1”中给出的与门源程序，编写一个异或门逻辑电路。用 Quartus II 波形仿真验证，下载到 DE0 开发板验证。
2. **要求 2:** 熟悉中规模器件译码器的逻辑功能，用硬件描述语言实现其设计。参考“参考内容 2”中给出的将 8421BCD 码转换成 0-9 的七段码译码器源程序，编写一个将二进制码转换成 0-E 的七段码译码器。用 Quartus II 波形仿真验证，下载到 DE0 开发板，利用开发板上的数码管验证。
3. **要求 3:** 熟悉时序电路计数器的逻辑功能，用硬件描述语言实现其设计。参考“参考内容 3”中给出的四位二进制计数器的源程序，编写一个计数器实现 0-E 计数。用 Quartus II 波形仿真验证。
4. **要求 4:** 熟悉分频电路的逻辑功能，并用硬件描述语言实现其设计。参考“参考内容 4”中给出的 50M 分频器的源程序，编写一个能实现占空比 50% 的 5M 和 50M 分频器即两个输出，输出信号频率分别为 10Hz 和 1Hz。下载到 DE0 开发板验证 (提示: 利用 DE0 板上已有的 50M 晶振作为输入信号，通过开发板上两个的 LED 灯观察输出信号)。
5. **要求 5:** 利用已经实现的 VHDL 模块文件，顶层文件采用原理图设计方法，实现 A 学号后 4 位 (3980)、F,E,E,L、学号后 4 位 (0938) 共 12 位字符在七段数码管上自动循环显示，频率 1Hz 和 10Hz 可以自动切换 (提示: 如何将 VHDL 模块文件在顶

层原理图文件中引用，参考参考内容 5)。

### 3 实验设备

- Quartus II 开发软件
- DE0 开发板 (FPGA 型号: Cyclone V 5CEBA4F23C7)
- USB-Blaster 下载线及计算机
- 七段数码管显示模块及相关输入、输出引脚

### 4 实验原理

#### 4.1 VHDL 基本结构

VHDL (Very High Speed Integrated Circuit Hardware Description Language) 是一种用于描述数字电路行为和结构的硬件描述语言。一个完整的 VHDL 设计文件由三部分组成：

1. **库引用**：通过 `library` 和 `use` 语句引入标准库。`ieee.std_logic_1164.all` 提供 `std_logic` 和 `std_logic_vector` 等标准逻辑类型；`ieee.numeric_std.all` 提供 `unsigned`、`signed` 等数值类型及算术运算支持。
2. **实体 (Entity)**：定义模块的外部接口，通过 `port()` 声明输入输出端口。端口类型包括 `std_logic` (单比特信号) 和 `std_logic_vector` (多比特总线)，方向有 `in`、`out`、`inout` 和 `buffer`。
3. **结构体 (Architecture)**：描述模块的内部行为或结构。可使用并发信号赋值语句实现组合逻辑，或使用 `process` 进程块实现时序逻辑。

#### 4.2 组合逻辑设计

组合逻辑电路的输出仅取决于当前输入，无记忆功能。在 VHDL 中，组合逻辑可通过以下两种方式实现：

- **并发信号赋值**：直接在结构体中使用赋值语句，如 `C <= A xor B;`，描述异或门逻辑。
- **进程中的 case 语句**：在 `process` 块中使用 `case` 语句进行多路选择，适用于译码器等多输入多输出的组合逻辑。

七段码译码器是典型的组合逻辑电路：根据 4 位 BCD 输入，通过 `case` 语句选择对应的 7 位段选码输出。DE0 开发板的七段数码管为共阳极结构，段选信号低电平有效（0 表示点亮，1 表示熄灭）。各数字对应的段选编码如表 1 所示。

表 1: 8421BCD 转七段码译码真值表

| BCD 输入 | 十六进制 | seg_out[6:0] | 显示字符 |
|--------|------|--------------|------|
| 0000   | 0    | 1000000      | 0    |
| 0001   | 1    | 1111001      | 1    |
| 0010   | 2    | 0100100      | 2    |
| 0011   | 3    | 0110000      | 3    |
| 0100   | 4    | 0011001      | 4    |
| 0101   | 5    | 0010010      | 5    |
| 0110   | 6    | 0000010      | 6    |
| 0111   | 7    | 1111000      | 7    |
| 1000   | 8    | 0000000      | 8    |
| 1001   | 9    | 0010000      | 9    |
| 1010   | A    | 0001000      | A    |
| 1011   | b    | 0000011      | b    |
| 1100   | C    | 1000110      | C    |
| 1101   | d    | 0100001      | d    |
| 1110   | E    | 0000110      | E    |
| 1111   | —    | 1111111      | 灭灯   |

### 4.3 时序逻辑设计

时序逻辑电路的输出不仅与当前输入有关，还与电路的历史状态有关，依赖时钟信号驱动状态变化。在 VHDL 中，时序逻辑通过 `process(clk, rst)` 进程块实现，使用 `rising_edge(clk)` 检测时钟上升沿来触发寄存器更新。

模  $N$  计数器是最基本的时序逻辑电路之一。以模 15 计数器为例，每个时钟上升沿计数值加 1，当计数到 14（即十六进制 E）后，下一个时钟回到 0，同时产生一个时钟周期宽

度的进位脉冲 COUT。计数器还具有异步清零功能：当复位信号 `rst` 为低电平时，计数值立即清零。

## 4.4 分频器原理

分频器通过对高频时钟信号进行计数来产生低频输出信号。若输入时钟频率为  $f_{\text{clk}}$ ，期望输出频率为  $f_{\text{out}}$ ，则需要的半周期计数值为

$$N = \frac{f_{\text{clk}}}{2 \times f_{\text{out}}}$$

计数器每计满  $N$  次，将输出信号翻转一次，即可产生占空比 50% 的方波。

对于 DE0 开发板上 50MHz 的晶振时钟：

- 分频到 1Hz:  $N = 50,000,000/2 = 25,000,000$
- 分频到 10Hz:  $N = 50,000,000/(2 \times 10) = 2,500,000$

## 4.5 综合设计原理

要求 5 的综合设计需要将计数器、译码器和分频/速度切换模块组合，实现 12 位字符的循环显示，并在 1Hz 和 10Hz 之间自动切换速度。

显示序列为 2539FEEL3980，由三部分组成：组员 B 学号后 4 位（2539）、固定字符 F,E,E,L、以及组员 A 学号后 4 位（3980）。M12 计数器从 0 计数到 11，每个计数值通过特殊七段码译码器映射到对应的显示字符，如表 2 所示。

表 2: 综合设计显示序列映射表

| 计数值 | BCD 码 | 显示字符 | SG_OUT[6:0] |
|-----|-------|------|-------------|
| 0   | 0000  | 2    | 0100100     |
| 1   | 0001  | 5    | 0010010     |
| 2   | 0010  | 3    | 0110000     |
| 3   | 0011  | 9    | 0010000     |
| 4   | 0100  | F    | 0001110     |
| 5   | 0101  | E    | 0000110     |
| 6   | 0110  | E    | 0000110     |
| 7   | 0111  | L    | 1000111     |
| 8   | 1000  | 3    | 0110000     |
| 9   | 1001  | 9    | 0010000     |
| 10  | 1010  | 8    | 0000000     |
| 11  | 1011  | 0    | 1000000     |

综合设计由三个核心模块组成：

1. **M12 计数器** (exa4\_3\_2): 0 到 11 循环计数, 带使能信号 **en**, 仅在 **en=1** 时计数加 1。计数到 11 后回到 0, 同时输出一个 **cout** 脉冲, 表示一轮显示结束。
2. **特殊七段码译码器** (exa4\_2\_2): 根据 M12 计数器的当前值, 输出对应字符的七段码。
3. **自动切换控制器** (exa4\_auto\_1hz\_10hz): 内部集成 1Hz 和 10Hz 两种分频模式。每当 M12 计数器完成一轮显示 (**cout=1**), 自动切换到另一种速度。其 **tick** 输出作为 M12 计数器的使能输入, 形成闭环控制。

顶层原理图中三个模块的连接关系为: 50MHz 时钟同时连接到自动切换控制器和 M12 计数器的时钟端; 复位信号同时连接到两者的复位端; 自动切换控制器的 **tick** 输出连接到 M12 计数器的 **en** 输入; M12 计数器的 **dout** 连接到译码器的 **BCD\_IN**; M12 计数器的 **cout** 反馈连接到自动切换控制器的 **loop\_end** 输入; 译码器的 **SG\_OUT** 输出连接到 HEX0 数码管引脚。

## 5 实验内容

### 5.1 要求 1: 异或门电路

#### 5.1.1 创建工程与编写 VHDL

打开 Quartus II, 选择 **File** → **New Project Wizard** 新建工程。器件选择 DE0 开发板对应的 Cyclone V 5CEBA4F23C7。选择 **File** → **New** → **VHDL File** 新建 VHDL 文件, 输入异或门源码并保存为 `exa4_1_xor.vhd`。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity exa4_1_xor is
5  port(
6      A : in  std_logic;
7      B : in  std_logic;
8      C : out std_logic
9  );
10 end exa4_1_xor;
11
12 architecture rtl of exa4_1_xor is
13 begin
14     C <= A xor B;
15 end rtl;
```

Listing 1: 异或门 VHDL 源码 (`exa4_1_xor.vhd`)

该设计定义了一个具有两个输入端 A、B 和一个输出端 C 的异或门, 结构体中使用并发信号赋值语句直接描述异或逻辑。

#### 5.1.2 编译与生成原理图框图

点击 **Start Compilation** 对 VHDL 文件进行完整编译。编译通过后, 选择 **File** → **Create/Update** → **Create Symbol Files for Current File**, 将 VHDL 模块生成可在原理

图中调用的模块符号文件 (.bsf)。

### 5.1.3 建立顶层原理图

选择 **File** → **New** → **Block Diagram/Schematic File** 新建原理图文件。在元件库的 Project 目录下找到刚生成的 `exa4_1_xor` 符号并放置到原理图中，然后添加输入引脚 A、B 和输出引脚 C，完成连线。顶层原理图如图 1 所示。

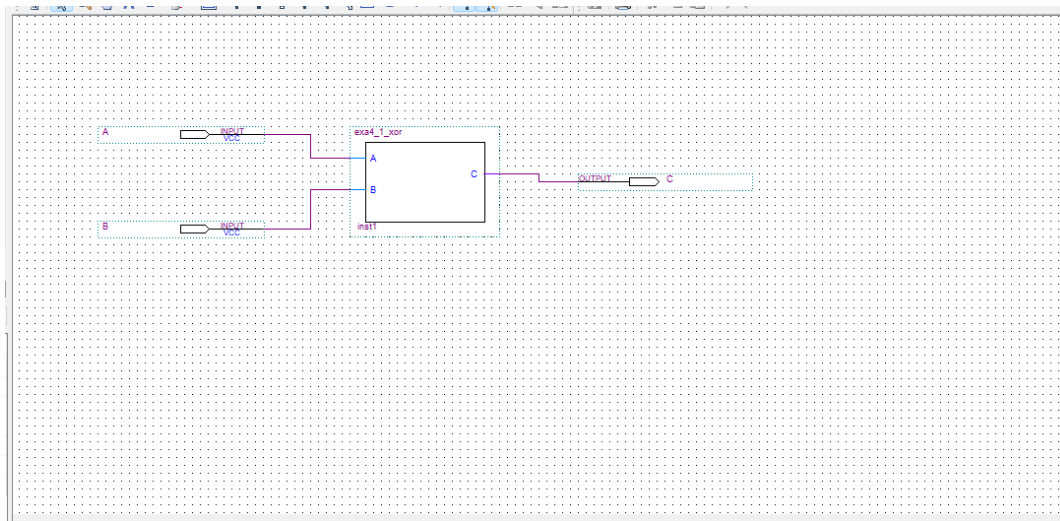


图 1: 要求 1 异或门 BDF 原理图

### 5.1.4 波形仿真验证

将原理图文件设置为顶层实体并重新编译。选择 **File** → **New** → **Vector Waveform File** 新建波形文件，加入 A、B、C 三个信号。设置 A 的周期为 20ns、B 的周期为 40ns，运行功能仿真。仿真结果如图 2 所示。



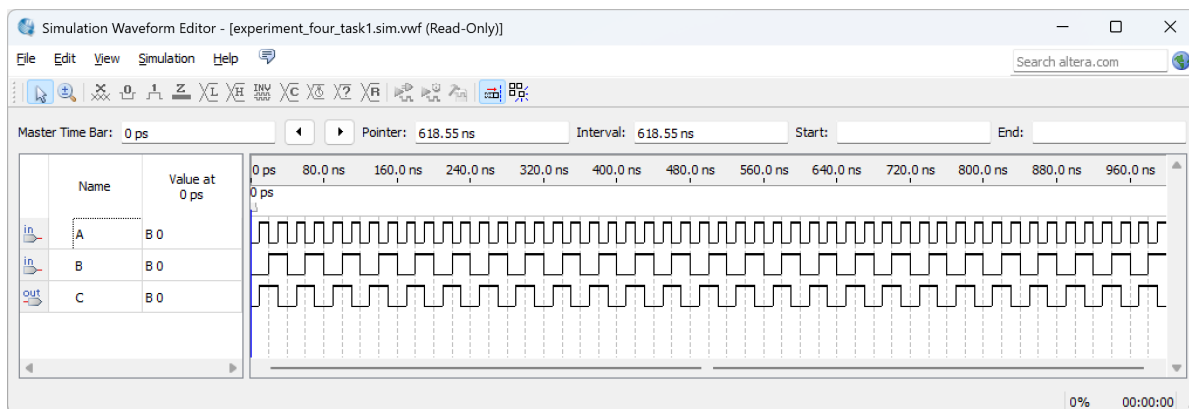


图 2: 要求 1 异或门仿真波形

从仿真波形可以看出, 当 A 和 B 的输入电平相同时, 输出 C 为低电平; 当 A 和 B 的输入电平不同时, 输出 C 为高电平, 符合异或门的逻辑功能。

### 5.1.5 引脚分配与下载验证

打开 Pin Planner, 将输入 A、B 分别分配到开发板的拨动开关 SW0 (PIN\_U13) 和 SW1 (PIN\_V13), 输出 C 分配到 LEDR0 (PIN\_AA2)。重新编译工程后, 连接 USB-Blaster, 选择 JTAG 模式下载 .sof 文件到 FPGA。在开发板上拨动开关, 观察 LED 指示灯正确反映异或逻辑。

## 5.2 要求 2: 8421BCD 转七段码译码器

### 5.2.1 创建工程与编写 VHDL

新建 Quartus II 工程, 创建 VHDL 文件 exa4\_2\_hex\_0e.vhd, 编写支持 0-E 共 15 个字符显示的七段码译码器。

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity exa4_2_hex_0e is
5 port(
6     data_in : in std_logic_vector(3 downto 0);
7     seg_out : out std_logic_vector(6 downto 0)
```

```
8 );
9 end exa4_2_hex_0e;
10
11 architecture rtl of exa4_2_hex_0e is
12 begin
13     process(data_in)
14     begin
15         case data_in is
16             when "0000" => seg_out <= "1000000"; -- 0
17             when "0001" => seg_out <= "1111001"; -- 1
18             when "0010" => seg_out <= "0100100"; -- 2
19             when "0011" => seg_out <= "0110000"; -- 3
20             when "0100" => seg_out <= "0011001"; -- 4
21             when "0101" => seg_out <= "0010010"; -- 5
22             when "0110" => seg_out <= "0000010"; -- 6
23             when "0111" => seg_out <= "1111000"; -- 7
24             when "1000" => seg_out <= "0000000"; -- 8
25             when "1001" => seg_out <= "0010000"; -- 9
26             when "1010" => seg_out <= "0001000"; -- A
27             when "1011" => seg_out <= "0000011"; -- b
28             when "1100" => seg_out <= "1000110"; -- C
29             when "1101" => seg_out <= "0100001"; -- d
30             when "1110" => seg_out <= "0000110"; -- E
31             when others => seg_out <= "1111111"; -- blank
32         end case;
33     end process;
34 end rtl;
```

Listing 2: 七段码译码器 VHDL 源码 (exa4\_2\_hex\_0e.vhd)

该译码器将 4 位 BCD 输入转换为 7 位段选信号输出。process 的敏感列表为 data\_in, 当输入变化时自动重新计算输出, 实现纯组合逻辑行为。段选码已按共阳极低电平有效编码, 可直接驱动 DE0 开发板的七段数码管。

### 5.2.2 编译、生成原理图框图与建立顶层原理图

编译 VHDL 文件后生成模块符号。新建 BDF 原理图，放置 `exa4_2_hex_0e` 符号，添加 4 位输入总线 `data_in[3..0]` 和 7 位输出总线 `seg_out[6..0]`，完成连线。顶层原理图如图 3 所示。

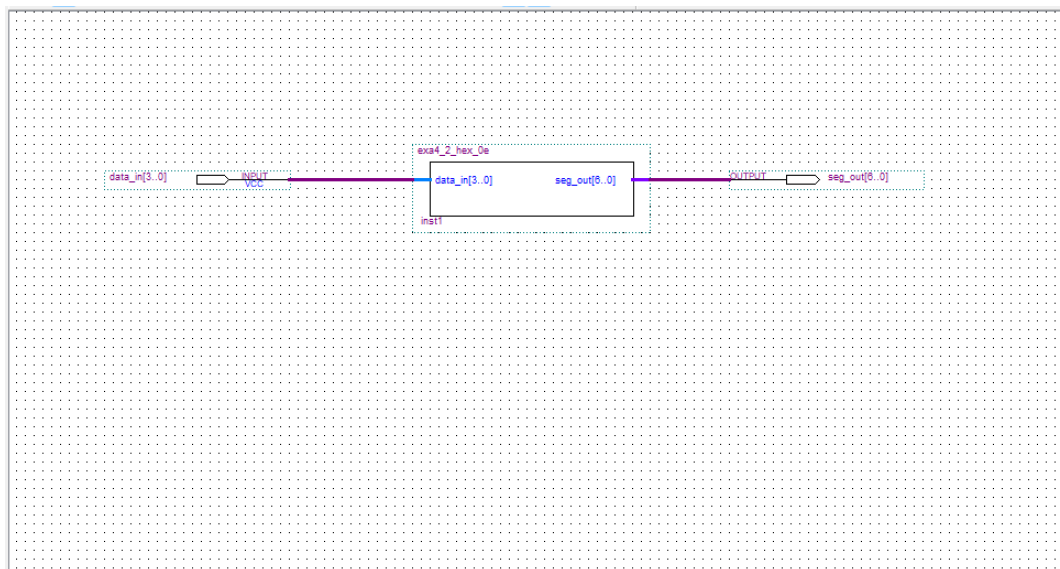


图 3: 要求 2 七段码译码器 BDF 原理图

### 5.2.3 波形仿真验证

建立波形文件，设置 `data_in` 依次从 0000 变化到 1111，每个输入保持一定时间。运行仿真后，检查 `seg_out` 的输出是否与表 1 中的真值表一致。仿真结果表明，16 种输入组合的段选输出均正确。

### 5.2.4 引脚分配与下载验证

将 `data_in[3:0]` 分配到拨动开关 SW3-SW0 (PIN\_T12、PIN\_T13、PIN\_V13、PIN\_U13)，`seg_out[6:0]` 分配到 HEX0 数码管的 7 个段选引脚 (PIN\_AA22、PIN\_Y21、PIN\_Y22、PIN\_W21、PIN\_W22、PIN\_V21、PIN\_U21)。重新编译并下载到 FPGA 后，拨动开关选择不同的 BCD 码，观察数码管正确显示 0-E 各字符。

## 5.3 要求 3: 模 15 计数器

### 5.3.1 创建工程与编写 VHDL

新建工程, 创建 VHDL 文件 exa4\_3\_counter\_0e.vhd, 编写模 15 计数器。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity exa4_3_counter_0e is
6  port(
7      clk : in std_logic;
8      rst : in std_logic;
9      DOUT : out std_logic_vector(3 downto 0);
10     COUT : out std_logic
11 );
12 end exa4_3_counter_0e;
13
14 architecture rtl of exa4_3_counter_0e is
15     signal count_q : unsigned(3 downto 0) := (others => '0');
16     signal cout_q : std_logic := '0';
17 begin
18     process(clk, rst)
19     begin
20         if rst = '0' then
21             count_q <= (others => '0');
22             cout_q <= '0';
23         elsif rising_edge(clk) then
24             if count_q = to_unsigned(14, 4) then
25                 count_q <= (others => '0');
26                 cout_q <= '1';
27             else
28                 count_q <= count_q + 1;
```

```

29         cout_q <= '0';
30     end if;
31 end if;
32 end process;
33
34 DOUT <= std_logic_vector(count_q);
35 COUT <= cout_q;
36 end rtl;

```

Listing 3: 模 15 计数器 VHDL 源码 (exa4\_3\_counter\_0e.vhd)

该计数器使用 `ieee.numeric_std` 库的 `unsigned` 类型进行计数，支持异步低电平复位。每个时钟上升沿计数加 1，当计数到 14 时下一个时钟回到 0 并产生一个周期的进位脉冲 COUT。

### 5.3.2 编译、生成原理图框图与建立顶层原理图

编译后生成模块符号。在顶层 BDF 中放置 `exa4_3_counter_0e` 符号，连接时钟和复位输入、计数输出和进位输出。引脚分配如表 3 所示，顶层原理图如图 4 所示。

表 3: 要求 3 引脚分配表

| 端口名     | 引脚      | 对应硬件     |
|---------|---------|----------|
| clk     | PIN_M9  | 50MHz 时钟 |
| rst     | PIN_U7  | KEY0     |
| DOUT[3] | PIN_Y3  | LEDR3    |
| DOUT[2] | PIN_W2  | LEDR2    |
| DOUT[1] | PIN_AA1 | LEDR1    |
| DOUT[0] | PIN_AA2 | LEDR0    |
| COUT    | PIN_N2  | LEDR4    |

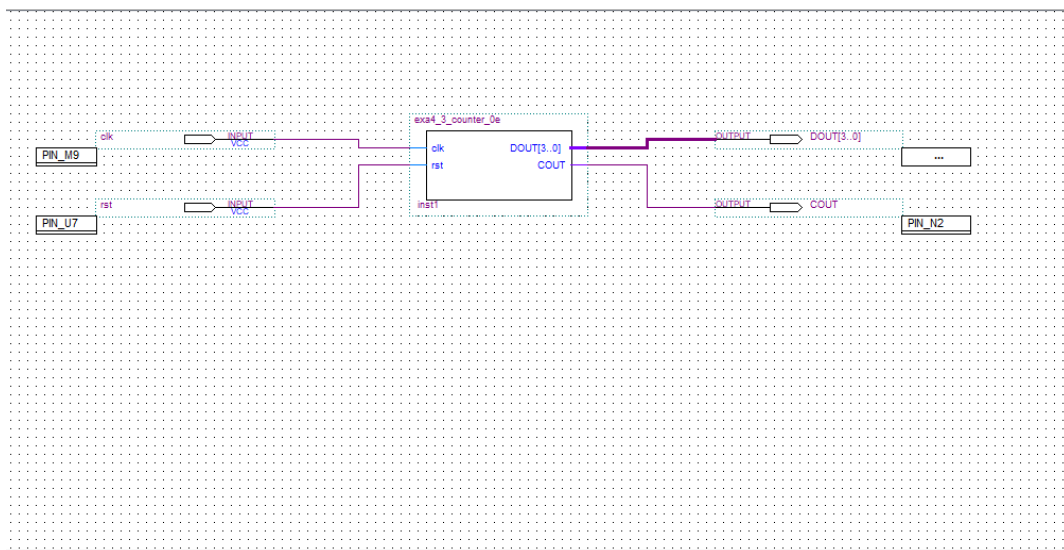


图 4: 要求 3 模 15 计数器 BDF 原理图

### 5.3.3 波形仿真验证

建立波形文件，设置时钟信号周期，在仿真开始时给复位端一个短暂的低电平脉冲。运行仿真后，观察 DOUT 依次输出 0, 1, 2, ..., 14, 0, 1, ... 的循环计数序列，COUT 在每次计数从 14 回到 0 时产生一个高电平脉冲，验证了计数器功能正确。

## 5.4 要求 4: 50M 分频器

### 5.4.1 创建工程与编写 VHDL

新建工程，创建 VHDL 文件 `exa4_4_divider_1hz_10hz.vhd`，编写同时输出 1Hz 和 10Hz 的双路分频器。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity exa4_4_divider_1hz_10hz is
5  generic(
6      HALF_PERIOD_1HZ  : positive := 25000000;
7      HALF_PERIOD_10HZ : positive := 2500000
8  );

```

```
9  port(  
10     clk50    : in  std_logic;  
11     clk_1hz   : out std_logic;  
12     clk_10hz  : out std_logic  
13 );  
14 end exa4_4_divider_1hz_10hz;  
15  
16 architecture rtl of exa4_4_divider_1hz_10hz is  
17     signal cnt_1hz   : integer range 0 to HALF_PERIOD_1HZ - 1 := 0;  
18     signal cnt_10hz  : integer range 0 to HALF_PERIOD_10HZ - 1 := 0;  
19     signal out_1hz   : std_logic := '0';  
20     signal out_10hz  : std_logic := '0';  
21 begin  
22     process(clk50)  
23     begin  
24         if rising_edge(clk50) then  
25             if cnt_1hz = HALF_PERIOD_1HZ - 1 then  
26                 cnt_1hz <= 0;  
27                 out_1hz <= not out_1hz;  
28             else  
29                 cnt_1hz <= cnt_1hz + 1;  
30             end if;  
31  
32             if cnt_10hz = HALF_PERIOD_10HZ - 1 then  
33                 cnt_10hz <= 0;  
34                 out_10hz <= not out_10hz;  
35             else  
36                 cnt_10hz <= cnt_10hz + 1;  
37             end if;  
38         end if;  
39     end process;  
40
```

```

41     clk_1hz <= out_1hz;
42     clk_10hz <= out_10hz;
43 end rtl;

```

Listing 4: 双路分频器 VHDL 源码 (exa4\_4\_divider\_1hz\_10hz.vhd)

分频器使用 `generic` 参数定义半周期计数值，使设计具有通用性。两路分频器共享同一个 `process`，各自独立计数。当计数达到半周期值时，输出信号翻转并重置计数，产生占空比 50% 的方波。

#### 5.4.2 编译、生成原理图框图与建立顶层原理图

编译后生成模块符号。在顶层 BDF 中放置分频器符号，连接输入时钟和两路输出。引脚分配如表 4 所示，顶层原理图如图 5 所示。

表 4: 要求 4 引脚分配表

| 端口名      | 引脚      | 对应硬件     |
|----------|---------|----------|
| clk50    | PIN_M9  | 50MHz 时钟 |
| clk_1hz  | PIN_W2  | LEDR2    |
| clk_10hz | PIN_AA1 | LEDR1    |



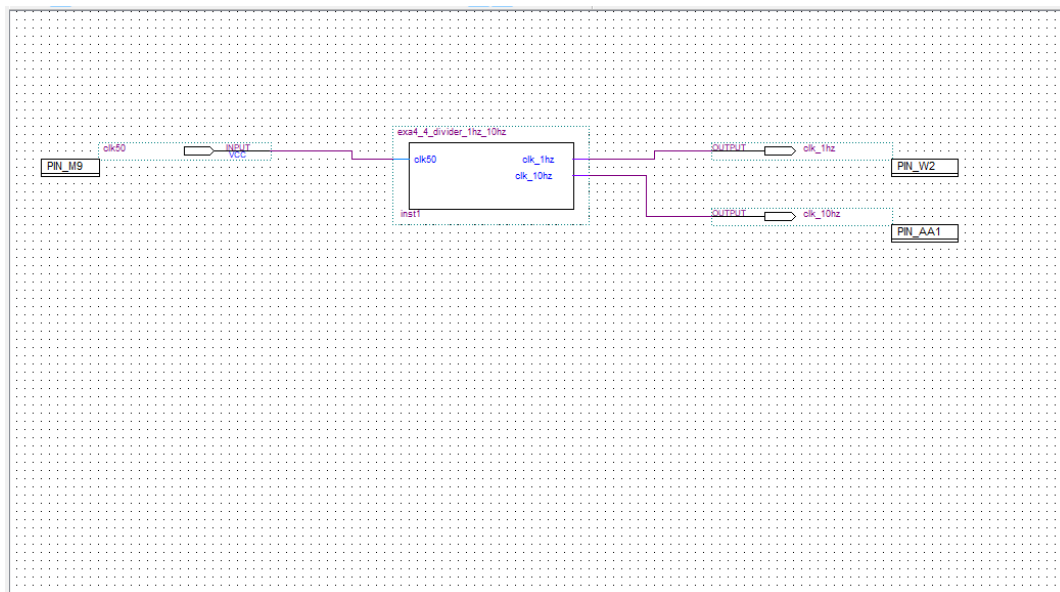


图 5: 要求 4 分频器 BDF 原理图

### 5.4.3 仿真验证与下载验证

由于使用实际分频参数（25,000,000 和 2,500,000）进行波形仿真需要极长的仿真时间，可通过编写测试台文件，使用 generic map 将半周期参数缩小（如设为 4 和 2），在较短的仿真时间内验证分频逻辑的正确性。

引脚分配完成后重新编译并下载到 DE0 开发板。利用板上 50MHz 晶振作为输入时钟，观察 LEDR2 以 1 秒为周期闪烁（1Hz），LEDR1 以 0.1 秒为周期闪烁（10Hz），验证分频器功能正确。

## 5.5 要求 5: 综合设计

### 5.5.1 设计方案

综合设计需要在一个七段数码管上循环显示 12 位字符 2539FEEL3980，并在每轮显示结束后自动切换 1Hz 和 10Hz 的显示速度。整个系统由三个核心 VHDL 模块和一个顶层 BDF 原理图组成。

### 5.5.2 编写 VHDL 模块

**特殊七段码译码器 (exa4\_2\_2)** 该译码器根据要求 5 的显示序列进行定制, 将计数值 0-11 映射到对应的七段显示码。

```
1  -- 七段码译码器: 显示 2539FEEL3980
2  library ieee;
3  use ieee.std_logic_1164.all;
4
5  entity exa4_2_2 is
6  port(
7      BCD_IN : in  std_logic_vector(3 downto 0);
8      SG_OUT : out std_logic_vector(6 downto 0)
9  );
10 end exa4_2_2;
11
12 architecture one of exa4_2_2 is
13 begin
14 process(BCD_IN)
15 begin
16     case BCD_IN is
17         when "0000" => SG_OUT <= "0100100"; -- 2
18         when "0001" => SG_OUT <= "0010010"; -- 5
19         when "0010" => SG_OUT <= "0110000"; -- 3
20         when "0011" => SG_OUT <= "0010000"; -- 9
21         when "0100" => SG_OUT <= "0001110"; -- F
22         when "0101" => SG_OUT <= "0000110"; -- E
23         when "0110" => SG_OUT <= "0000110"; -- E
24         when "0111" => SG_OUT <= "1000111"; -- L
25         when "1000" => SG_OUT <= "0110000"; -- 3
26         when "1001" => SG_OUT <= "0010000"; -- 9
27         when "1010" => SG_OUT <= "0000000"; -- 8
28         when "1011" => SG_OUT <= "1000000"; -- 0
```

```
29         when others => SG_OUT <= "1111111";
30     end case;
31 end process;
32 end one;
```

Listing 5: 特殊七段码译码器 (exa4\_2\_2.vhd)

**M12 计数器 (exa4\_3\_2)** M12 计数器在模 15 计数器基础上修改为 0~11 循环, 并新增使能信号 en, 仅在使能有效时计数。

```
1  -- M12 计数器: 0~11 循环, 用于显示 2539FEEL3980 共 12 位
2  library ieee;
3  use ieee.std_logic_1164.all;
4  use ieee.numeric_std.all;
5
6  entity exa4_3_2 is
7  port(
8      clk  : in  std_logic;
9      rst  : in  std_logic;
10     en   : in  std_logic;
11     dout : out std_logic_vector(3 downto 0);
12     cout : out std_logic
13 );
14 end exa4_3_2;
15
16 architecture one of exa4_3_2 is
17     signal Q1 : unsigned(3 downto 0) := (others => '0');
18 begin
19
20     process(clk, rst)
21     begin
22         if rst = '0' then
```

```

23     Q1 <= (others => '0');
24     cout <= '0';
25
26     elsif rising_edge(clk) then
27         cout <= '0';
28
29         if en = '1' then
30             if Q1 = to_unsigned(11, 4) then
31                 Q1 <= (others => '0');
32                 cout <= '1';
33             else
34                 Q1 <= Q1 + 1;
35             end if;
36         end if;
37     end if;
38 end process;
39
40 dout <= std_logic_vector(Q1);
41
42 end one;

```

Listing 6: M12 计数器 (exa4\_3\_2.vhd)

**自动 1Hz/10Hz 切换控制器 (exa4\_auto\_1hz\_10hz)** 该模块集成了分频和速度切换功能。内部维护一个模式标志 `mode` (0 表示 1Hz, 1 表示 10Hz), 每当收到 M12 计数器的 `loop_end` 信号时切换模式。根据当前模式选择不同的分频系数, 产生对应频率的 `tick` 脉冲。

```

1  -- 1Hz / 10Hz 自动切换 tick 发生器
2  library ieee;
3  use ieee.std_logic_1164.all;
4

```

```
5  entity exa4_auto_1hz_10hz is
6  port(
7      clk50      : in  std_logic;
8      rst        : in  std_logic;
9      loop_end    : in  std_logic;
10     tick        : out std_logic;
11     speed_sel   : out std_logic
12 );
13 end exa4_auto_1hz_10hz;
14
15 architecture rtl of exa4_auto_1hz_10hz is
16
17     constant DIV_1HZ  : integer := 50000000;
18     constant DIV_10HZ : integer := 5000000;
19
20     signal cnt        : integer range 0 to DIV_1HZ - 1 := 0;
21     signal mode       : std_logic := '0';
22     signal tick_r     : std_logic := '0';
23
24     begin
25
26     process(clk50, rst)
27         variable limit_v : integer;
28     begin
29         if rst = '0' then
30             cnt      <= 0;
31             mode     <= '0';
32             tick_r <= '0';
33
34         elsif rising_edge(clk50) then
35             tick_r <= '0';
36
```

```
37     if loop_end = '1' then
38         mode <= not mode;
39         cnt  <= 0;
40
41     else
42         if mode = '0' then
43             limit_v := DIV_1HZ - 1;
44         else
45             limit_v := DIV_10HZ - 1;
46         end if;
47
48         if cnt >= limit_v then
49             cnt  <= 0;
50             tick_r <= '1';
51         else
52             cnt <= cnt + 1;
53         end if;
54     end if;
55 end if;
56 end process;
57
58 tick <= tick_r;
59 speed_sel <= mode;
60
61 end rtl;
```

Listing 7: 自动 1Hz/10Hz 切换控制器 (exa4\_auto\_1hz\_10hz.vhd)

### 5.5.3 生成原理图框图

对上述三个 VHDL 模块分别进行编译, 然后通过 **File** → **Create/Update** → **Create Symbol Files for Current File** 生成各自的 .bsf 模块符号文件, 使其可在顶层原理图中

作为元件调用。

#### 5.5.4 建立顶层原理图与引脚分配

新建 BDF 原理图文件作为顶层设计，从 Project 元件库中依次放置三个模块符号，按照以下连接关系完成布线：

- 50MHz 时钟 (PIN\_M9) 同时连接到 exa4\_auto\_1hz\_10hz 的 clk50 端和 exa4\_3\_2 的 clk 端。
- 复位信号 (PIN\_U13, 按键 SW0) 同时连接到两个模块的 rst 端。
- exa4\_auto\_1hz\_10hz 的 tick 输出连接到 exa4\_3\_2 的 en 输入。
- exa4\_3\_2 的 dout[3:0] 连接到 exa4\_2\_2 的 BCD\_IN[3:0]。
- exa4\_3\_2 的 cout 反馈连接到 exa4\_auto\_1hz\_10hz 的 loop\_end 输入。
- exa4\_2\_2 的 SG\_OUT[6:0] 连接到 HEX0 数码管输出引脚。

引脚分配如表 5 所示，顶层原理图如图 6 所示。

表 5: 要求 5 引脚分配表

| 端口名               | 引脚       | 对应硬件     |
|-------------------|----------|----------|
| clk50 (pin_name4) | PIN_M9   | 50MHz 时钟 |
| rst (pin_name5)   | PIN_U13  | SW0      |
| SG_OUT[6]         | PIN_AA22 | HEX0[6]  |
| SG_OUT[5]         | PIN_Y21  | HEX0[5]  |
| SG_OUT[4]         | PIN_Y22  | HEX0[4]  |
| SG_OUT[3]         | PIN_W21  | HEX0[3]  |
| SG_OUT[2]         | PIN_W22  | HEX0[2]  |
| SG_OUT[1]         | PIN_V21  | HEX0[1]  |
| SG_OUT[0]         | PIN_U21  | HEX0[0]  |

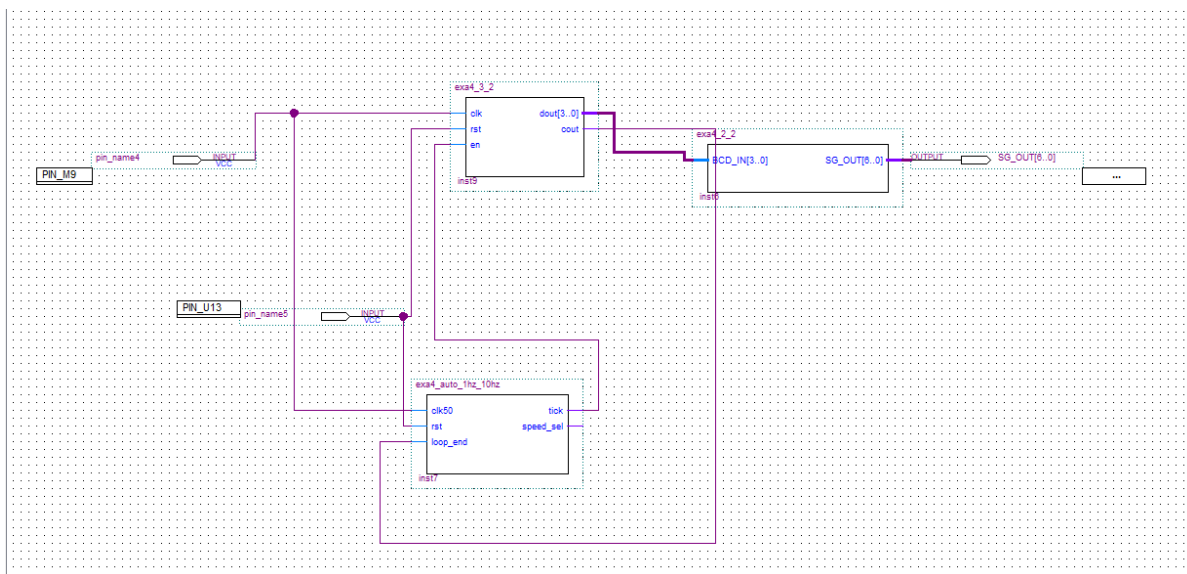


图 6: 要求 5 综合设计 BDF 原理图

### 5.5.5 编译与下载验证

将 BDF 原理图文件设置为顶层实体, 进行完整编译。确认编译无错误后, 通过 USB-Blaster 以 JTAG 模式下载到 DE0 开发板。

下载完成后, 观察 HEX0 数码管的显示行为: 数码管以 1Hz 的速度依次显示 2, 5, 3, 9, F, E, E, L, 3, 9, 8, 0 共 12 个字符, 完成一轮后自动切换到 10Hz 速度进行下一轮循环显示, 然后再切回 1Hz, 如此交替进行。按下复位键可使显示从第一个字符重新开始。

## 6 实验过程中的问题

1. **VHDL 模块生成原理图框图:** 本实验首次使用 VHDL 描述电路模块。初次操作时不清楚如何将 VHDL 文件变为可在 BDF 原理图中调用的元件。关键步骤是编译 VHDL 文件后, 选择 File → Create/Update → Create Symbol Files for Current File 生成 .bsf 符号文件。若跳过此步骤, 在元件库中将找不到对应模块。
2. **分频器仿真时间过长:** 使用实际半周期参数 (25,000,000) 进行波形仿真时, 即使只观察一个完整周期也需要模拟 5000 万个时钟周期, 仿真耗时极长。解决方法是编写测试台文件, 通过 generic map 将半周期参数缩小为较小的值 (如 4 和 2), 在合理的仿真时间内验证分频逻辑是否正确。



3. **七段数码管有效电平**: DE0 开发板的七段数码管为共阳极结构, 段选信号低电平有效 (输出 0 点亮对应段, 输出 1 熄灭对应段)。若译码器的段选编码极性错误, 数码管上所有笔段会出现反转显示。本实验的 VHDL 代码中已按低电平有效编码, 可直接驱动数码管。
4. **综合设计中模块间的反馈连接**: 要求 5 中, M12 计数器的 `cout` 输出需反馈到自动切换控制器的 `loop_end` 输入, 而控制器的 `tick` 输出又连接到计数器的 `en` 输入, 形成闭环。若任一反馈连线缺失, 系统将无法正常工作: 缺少 `tick` 连接时计数器不会计数, 缺少 `cout` 反馈时速度无法切换。

## 7 心得体会

1. 通过本次实验, 我初步掌握了 VHDL 硬件描述语言的基本语法和设计方法。与前几次实验中使用的原理图输入方式相比, VHDL 能够用简洁的文字代码描述复杂的逻辑功能, 尤其在实现译码器、计数器等功能模块时, 代码方式比手工连线更加高效且不易出错。
2. 本实验从简单的异或门出发, 逐步过渡到译码器、计数器、分频器, 最终完成多模块综合设计, 让我深刻体会到模块化设计的思想。每个功能模块可以独立设计、独立仿真验证, 确认正确后再在顶层进行集成, 大大降低了复杂系统的调试难度。
3. 要求 5 的综合设计将 VHDL 文本描述与 BDF 图形化连接相结合, 发挥了两种设计方法各自的优势: VHDL 适合描述功能逻辑, BDF 原理图适合展示模块间的连接关系和信号流向。这种混合设计方法在实际的 FPGA 工程开发中也是常用的实践方式。